

MATH 345 Skeleton Notes

Unit 1: Vectors, matrices, and linear maps

Section 1.5: Coding recap

© 2024–2026 Jacob Denson, Kaitlyn Phillipson, Sebastien Roch. Except where otherwise noted, this work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Linked notebook

Run [Lab U1: Vectors, similarity, attention, and matrix actions](#). The main sequence practices vector applications, document similarity, token attention, matrix-vector products, geometric matrix actions, applying one matrix to many points, composition, affine maps, and short code outputs. The final part contains review questions and a short forward look to square-grid visualizations.

For a quick reference on arrays, matrix products, norms, rankings, and slicing, see the programming appendix sections NumPy Arrays, Vectors, Matrices, and Shapes, Elementwise Arithmetic Versus Linear Algebra, Dot Products, Norms, Distances, and Cosine Similarity, Sorting and Rankings, and Indexing and Slicing.

Review activities

Activity: Reading a ranking (U1-LO2, U1-LO8)

Continuing the cosine-similarity ranking from The same rankings in code (U1-LO2, U1-LO8), the code is

```
scores = np.array([1.0, 0.5, 0.0])
doc_names = np.array(["D1", "D2", "D3"])
ranking = doc_names[np.argsort(scores)[::-1]]
ranking
```

1. What is the value of ranking?
2. What is being ranked?
3. Is this ranking based on distance or direction?

Activity: Reading an attention computation (U1-LO4, U1-LO8)

In The same attention calculation in code (U1-LO4, U1-LO8), the code was

```
s = K @ q  
alpha = s / s.sum()  
out = alpha @ V
```

1. What does s contain?
2. Why do the entries of alpha add to 1?
3. What does out represent?

Activity: Reading a geometric computation (U1-LO5, U1-LO8)

In Applying one matrix to many points (U1-LO5, U1-LO8), the code was

```
X = np.array([
    [0, 1, 1, 0, 0],
    [0, 0, 1, 1, 0],
], dtype=float)
```

```
A = np.array([
    [1, 1],
    [0, 1],
], dtype=float)
```

```
Y = A @ X
Y
```

1. What are the columns of X ?
2. What are the columns of Y ?
3. Which transformation from the gallery is this?

Activity: Reading composition order (U1-L06)

```
S = np.array([
    [2., 0.],
    [0., 1.],
])

R = np.array([
    [0., -1.],
    [1., 0.],
])

R @ S, S @ R
```

1. Which product applies S first and then R?
2. Are the two products equal?
3. What does the difference mean geometrically?

Activity: Reading an affine rule (U1-LO7)

```
W = np.array([
    [1., 1.],
    [0., 2.],
])

b = np.array([3., -1.])

def f(x):
    return W @ x + b
```

Is f linear? Justify your answer using the zero vector.